IEEE Elevate: Project Hub

Project Descriptions by Track

Contents

1. Software Engineering Track

Project 1: Content Management System

Target Audience: Intermediate students with a solid grasp of web fundamentals, ready to tackle complex architecture, relational data modeling, and performance optimization.

The Problem: Organizations and content teams need a reliable way to create, manage, and publish digital content with multiple contributors, different permission levels, and a growing media library, without relying on rigid or expensive third-party platforms.

Features:

- Role-based access control (Admin, Editor, Contributor, Viewer).
- Dynamic article creation with a rich-text (WYSIWYG) editor and draft/publish states.
- Media upload pipeline for image and file management.
- RESTful API endpoints to serve content to external clients.

Skills:

- Advanced database schema design and querying.
- Implementing secure authentication and authorization protocols (e.g., JWT or session-based).
- Building scalable API architecture and handling background processing for heavy tasks like image resizing.
- Writing comprehensive system documentation.

Technologies:

- Python, Django, PostgreSQL
- Celery & RabbitMQ (for asynchronous tasks)
- Frontend framework (React or Vue.js) or robust Django templates
- Cloud storage integration (e.g., AWS S3 for media)

Project 2: Polling & Survey App

Target Audience: Beginners transitioning to intermediate. They understand syntax and basic routing but need practice integrating different parts of a web application and managing state.

The Problem: Teams, educators, and event organizers often need a fast way to collect opinions or feedback from a group and see the results visualized in real time, without duplicate or manipulated submissions.

Features:

- User-generated surveys with multiple question types (multiple choice, text input).
- Secure voting mechanisms to prevent duplicate submissions.
- Real-time dashboard with charts displaying survey results.
- Shareable, unique URLs for individual polls.

Skills:

- Mastering CRUD (Create, Read, Update, Delete) operations.
- Designing one-to-many and many-to-many database relationships.
- Integrating third-party JavaScript libraries for data visualization.
- Handling form validations and secure data submission.

Technologies:

- Python, Django, SQLite (or PostgreSQL)
- HTML, CSS, JavaScript
- Chart.js or D3.js (for analytics rendering)
- Tailwind CSS or Bootstrap

Project 3: Habit Tracker

Target Audience: True beginners building their first complete full-stack application. The focus is on core web development concepts, basic logic, and successful deployment.

The Problem: New developers need a simple, complete project that lets them practice the full web development cycle end-to-end, while also giving users a practical tool to build and maintain daily habits.

Features:

- Simple user registration and login.

- Dashboard to add, edit, or delete daily habits.
- A daily check-in system with a basic streak counter.
- Progress tracking showing completion percentages over the week.

Skills:

- Understanding the request-response cycle and basic routing.
- Implementing fundamental database queries.
- Working with date and time logic in code.
- Structuring clean HTML/CSS (ensuring semantic best practices, like keeping anchor tags separate from button tags for actions).

Technologies:

- Python, Django (or Flask for a lighter footprint)
- SQLite
- Vanilla JavaScript, HTML5, CSS3

2. Cyber Security Track

Project 1: Password Strength & Breach Checker

Target Audience: Complete beginners to cybersecurity building their first hands-on project. The focus is on core security concepts and putting together a small, complete tool without unnecessary complexity.

The Problem: Many users rely on weak or previously leaked passwords without realizing it, leaving their accounts vulnerable to credential-stuffing and brute-force attacks.

Features:

- A password strength meter based on length and character variety.
- Checks whether a password has appeared in known data breaches (Have I Been Pwned API).
- A simple summary report highlighting weaknesses and improvement tips.
- A simple CLI interface, or an optional small web page.

Skills:

- Understanding hashing fundamentals (e.g., SHA-1) and how they protect data.
- Safely integrating with external APIs (K-Anonymity model).
- Understanding core concepts like entropy, brute-force attacks, and dictionary attacks.
- Writing clean, well-organized Python code.

Technologies:

- Python
- hashlib, requests
- (Optional) Flask for a simple web interface

Project 2: Mini Network Vulnerability Scanner

Target Audience: Trainees with solid programming basics who have covered an introduction to networking, ready to apply what they've learned in a tool that actually interacts with a network.

The Problem: Organizations often run outdated or misconfigured services on open network ports without realizing they are exposed to known vulnerabilities.

Features:

- Scans for open ports (Port Scanning) on a target host or IP range.
- Identifies the services running on each open port (Service/Version Detection).
- Basic comparison against a known vulnerability database (e.g., CVE) for outdated services.
- A final report (HTML or PDF) summarizing open ports and potential risks.

Skills:

- Understanding TCP/IP fundamentals and common ports (HTTP, FTP, SSH...).
- Socket programming in Python.
- Reading and understanding basic vulnerability reports.
- Working with existing network-scanning libraries such as nmap.

Technologies:

- Python
- python-nmap or the core sockets library
- (Optional) Scapy for packet analysis
- HTML for displaying the final report

Project 3: Simple SIEM – Log Analysis & Intrusion Detection Dashboard

Target Audience: Beginners who have reached a good level and are ready for a more complete project that combines data analysis with security detection logic, without being overly complex.

The Problem: Security teams need to quickly detect suspicious activity, such as brute-force login attempts or injection attacks, buried within large volumes of raw log data.

Features:

- Reads and parses log files (e.g., auth.log or web server logs).
- Detects suspicious patterns, such as repeated failed login attempts (brute force) or SQL injection attempts.
- A simple dashboard that displays alerts in real time or near real time.
- A basic simulation of blocking a suspicious IP (IP blacklisting).

Skills:

- Parsing large text files using regex and basic data processing.
- Understanding and writing simple detection rules.
- Recognizing common attack signatures.
- Building a simple interactive dashboard that presents results clearly.

Technologies:

- Python, Pandas
- Flask or Django for the dashboard
- Chart.js for data visualization
- SQLite for storing events and alerts

3. Entrepreneurship Track

Project 1: Viral Waitlist and Referral API

Target Audience: Intermediate students comfortable with backend development, ready to build a product-focused service that supports early-stage startup validation.

The Problem: Early-stage founders need to validate their ideas and build an audience before they write complex product code.

Features:

- Reusable, embeddable backend service for capturing waitlist signups.
- Unique referral link generation for each user.
- Referral tracking that moves users up the queue based on signups they bring in.
- Email notifications confirming signup and referral progress.

Skills:

- API development and endpoint design.
- Designing database relations to track users and referrals.
- Setting up email automation workflows.
- Generating and validating unique tokens/links securely.

Technologies:

- Python (Django/Flask) or Node.js backend
- PostgreSQL or MongoDB
- SendGrid or Mailgun (email automation)
- JWT or UUID-based token generation

4. Data Science Track

Project 1: Automated Exploratory Data Analysis (EDA) Generator

Target Audience: Beginners in data science comfortable with Python basics, ready to practice data cleaning and automated reporting.

The Problem: Data scientists spend too much time writing the same boilerplate code to understand the basic shape of a new dataset.

Features:

- Accepts any tabular dataset (CSV/Excel) as input.
- Automatically generates a report detailing missing values and data distributions.
- Outlier detection across numerical columns.
- Correlation matrix visualization between variables.

Skills:

- Data cleaning and preprocessing with Pandas.
- Statistical summarization and outlier detection techniques.
- Generating visualizations for data distributions and correlations.
- Automated report generation (HTML/PDF).

Technologies:

- Python, Pandas
- Matplotlib/Seaborn
- Automated reporting (ReportLab or Pandas Profiling)

Project 2: Interactive Customer Churn Prediction App

Target Audience: Intermediate students with a grasp of Python and basic machine learning concepts, ready to build and deploy a predictive model.

The Problem: Businesses need to know which customers are likely to cancel their subscriptions so they can intervene proactively.

Features:

- ML model trained on a public customer churn dataset.

- Interactive web app where stakeholders can adjust customer parameters.
- Instant churn probability prediction based on input values.
- Model explainability visualizations showing key churn drivers.

Skills:

- Feature engineering for structured/tabular data.
- Training and evaluating classification algorithms.
- Model explainability using SHAP values.
- Building interactive apps with Streamlit or Gradio.

Technologies:

- Python, Scikit-learn (Random Forest or XGBoost)
- SHAP (model explainability)
- Streamlit or Gradio

5. Artificial Intelligence Track

Project 1: AI-Powered Autonomous Vehicle Collision Risk Predictor

Target Audience: Intermediate learners interested in advanced ADAS (Advanced Driver Assistance Systems), autonomous driving logic, and merging visual systems, tabular ML modeling, and Large Language Model (LLM) APIs.

The Problem: Modern autonomous vehicles must continuously process complex physical environments and instantly react to threats. While computer vision models can count obstacles, they do not inherently understand how the relationship between current speed and obstacle density impacts hazard levels. Operators and fleet managers need a unified pipeline that detects obstacles, uses tabular ML to predict safety risk levels, and generates structured driving hazard reports automatically.

Features:

- **Visual Obstacle Detection (CV/DL):** Ingests an image from the vehicle's dashboard camera to detect and count obstacles (such as other cars, pedestrians, traffic cones, and barriers) using an object detection model (like YOLOv8).
- **Data Fusion & Logging:** Extracts the vehicle's telemetry speed at that exact second (passed as API metadata or via OCR) and saves it alongside the visual obstacle count in a database table.
- **Risk Prediction Engine (Tabular ML):** Uses the saved structured values (current_speed and obstacle_count) to predict a Vehicle Safety Level (Safe, Alert, Critical) or recommend a Safe Velocity Adjustment using a decision tree or random forest model.
- **Gemini Copilot API Integration:** Sends the numerical data (speed, obstacles detected) and the ML prediction (Safety Level) to the Google Gemini API to generate a structured, human-readable safety incident report and corrective driving recommendations.
- **Unified REST API:** Exposes a POST /analyze-scene endpoint using FastAPI that accepts an image file and the speed telemetry, orchestrates the entire workflow, and returns the combined ML and Gemini JSON response.

Skills:

- **Computer Vision:** Utilizing state-of-the-art architectures (YOLOv8) to isolate, label, and tally specific visual objects in a scene.

- **Tabular Machine Learning:** Training and deploying a scikit-learn classification/regression model utilizing engineered physical parameters (like relative distance assumptions based on speed and count).
- **Generative AI Engineering:** Writing structured prompts and implementing the unified google-genai Python SDK to call Gemini models (e.g., gemini-2.5-flash or gemini-1.5-flash) for real-time natural language reporting.
- **Web Services & API Design:** Building a secure API gateway that coordinates file uploads, runs two different models sequentially, and integrates third-party APIs asynchronously.

Technologies:

- Python
- OpenCV & PyTorch / Ultralytics YOLOv8 (for obstacle detection)
- Scikit-Learn (for tabular classification/regression of safety levels)
- google-genai (Google's Gen AI SDK for the Gemini API)
- FastAPI & Uvicorn (for API routing and server serving)
- SQLite or Pandas (for logging speed and obstacle metrics)

Project 2: AI-Powered Resume & Job Matching Tool

Target Audience: Intermediate students comfortable with APIs and basic backend logic, ready to combine NLP techniques with practical scoring and ranking systems.

The Problem: Recruiters and job seekers struggle to quickly assess how well a resume matches a job description, leading to missed opportunities or overlooked qualified candidates.

Features:

- Upload and parse resumes (PDF/DOCX) to extract skills, experience, and education.
- Compare parsed resume content against a job description using semantic similarity.
- Generate a match score with specific improvement suggestions (missing skills, keyword gaps).
- Dashboard displaying ranked candidates for a given job posting.

Skills:

- Text extraction and preprocessing from unstructured documents.
- Generating and comparing embeddings for semantic similarity.
- Designing scoring/ranking algorithms.
- Handling file uploads and parsing securely.

Technologies:

- Python, spaCy or NLTK (text processing)
- OpenAI/embedding models or Sentence-Transformers
- FAISS or ChromaDB (similarity search)
- Flask or Streamlit (interface)

Project 3: Multi-Agent AI Research & Report Generator

Target Audience: Advanced students with a solid grasp of AI fundamentals, ready to design multi-step agentic systems that coordinate several AI components together.

The Problem: Producing a well-researched, structured report on a topic typically requires manually searching, summarizing, writing, and reviewing content, a process that is slow and hard to scale.

Features:

- User submits a research topic or question.
- Multiple specialized agents (searcher, summarizer, writer, reviewer) collaborate in sequence to produce a structured report.
- Iterative refinement loop where the reviewer agent flags gaps and sends content back for revision.
- Exportable final report (PDF or Markdown) with cited sources.

Skills:

- Designing and orchestrating multi-agent pipelines.
- Prompt chaining and structured tool/function calling.
- Managing asynchronous, multi-step AI workflows.
- Output validation and quality control for AI-generated content.

Technologies:

- Python, LangChain, CrewAI, or AutoGen (agent orchestration)
- OpenAI API or an open-source LLM
- FastAPI (backend)
- ReportLab or WeasyPrint (PDF generation)